NoteGuy

An intelligent markdown workspace

LightRAG-Powered Knowledge Graph Retrieval

WYSIWYG Markdown Editing with Folder Organization

Git-Backed Version History with Diff & Restore

Multimodal Document Ingestion via RAG-Anything

AI Chat, Extraction, & Deep Analysis Over Your Vault

Runtime-Configurable AI Settings

TECHNOLOGY STACK

FastAPI · React 19 · TypeScript · Tailwind CSS · Vite

Dashboard

The NoteGuy dashboard provides a unified command center for your workspace. Quickly create notes, upload documents, and access the full suite of AI tools — all with real-time knowledge graph statistics at a glance.

AI Tools
Chat, Query, Extract, Analyze, and Ingest

Knowledge Graph
Live entity & relation counts at a glance

Drag & Drop
Upload documents directly for multimodal ingestion

WYSIWYG Markdown Editor

Write and organize notes with a rich WYSIWYG editor powered by Milkdown. Folder-based organization, block quotes, headings, and inline formatting — all rendered in a clean, distraction-free interface.

Folder Tree Hierarchical sidebar for project-based organization

Rich Editing Headings, block quotes, lists, and inline styles

Git History Every edit is versioned — diff and restore any revision

Knowledge Graph

NoteGuy automatically builds a knowledge graph from your notes using LightRAG. Entities, relationships, and concepts are extracted and visualized as an interactive, color-coded graph — enabling powerful cross-document retrieval.

Full knowledge graph — entities colored by type, sized by connection density

Graph Detail View

Click any entity to explore its relationships and connections.
The detail view highlights linked nodes and surfaces entity descriptions,
making it easy to trace how concepts relate across your entire vault.

Entity detail — selecting "FastAPI Backend" reveals its direct relationships

Architecture Overview

Data Flow

1. User edits a note in the WYSIWYG editor
2. Backend persists to SQLite + writes markdown to the vault
3. Ingestion tracker marks the note dirty for deferred indexing
4. Git service commits the change for version history
5. On next AI query, dirty notes are flushed into the knowledge graph
6. Chat and AI routes query LightRAG with the configured models
7. Streaming responses are delivered to the UI via SSE

Runtime Components

FastAPI Backend	API surface, CRUD, ingestion, RAG chat, history, settings
React Frontend	Editor, sidebar, chat panel, AI tools, settings views
LightRAG	Knowledge graph build, hybrid query, extraction, analysis
RAG-Anything	Multimodal document parsing (PDF, PPTX, images)
SQLite	Note and folder metadata, app state
Git Service	Versioning, diffs, and restore for note files
Tauri Shell	Optional native desktop packaging

Open Source on GitHub

github.com/chaserwacer/noteguy

MIT License · Python 3.11+ · Node 18+